

1. Real-World Problem Statement

Modern vehicles generate continuous sensor data — engine temperature, oil pressure, battery health, brake wear, load patterns — but this data is processed almost entirely through static, threshold-based logic. A warning triggers only when a single value crosses a fixed limit. This creates two distinct, equally significant gaps: one in routine, everyday vehicle management, and one in rare, high-stakes failure detection.

Gap 1 — Everyday vehicle intelligence is generic and reactive. Maintenance is scheduled on fixed mileage or time intervals, regardless of how a vehicle is actually driven — a car used mostly for short city trips and one used for highway cruising receive identical service recommendations, even though their real wear patterns differ substantially. Drivers get no meaningful feedback connecting their day-to-day driving behavior to fuel/energy efficiency or long-term component wear. Dashboard warnings, when they do appear, are opaque — a driver sees a code or a light with no explanation of why it triggered or what it means in the context of their recent driving. As a result, most drivers interact with their vehicle's intelligence only when something is already wrong, not as part of routine ownership. This is a daily, universal problem — it affects every driver, every trip, regardless of geography or connectivity.

Gap 2 — Rare, compounding failures are caught too late. Real mechanical failures rarely announce themselves through one sensor crossing one threshold; they emerge from combinations of individually minor anomalies — a slight temperature rise, a subtle pressure drop, an unusual load pattern — that no single rule is designed to catch together. By the time a threshold-based system reacts, the failure is often already acute. This risk is amplified in low-connectivity environments — rural highways, mountain routes, mining and construction sites — where cloud-based analytics are unavailable exactly when they would matter most.

Current production vehicles rely predominantly on rule-based diagnostics and component-specific analytics for both of these situations. A continuously reasoning, edge-first architecture that addresses routine usage-based intelligence and rare compounding-failure detection through one system remains uncommon in publicly documented automotive offerings — most existing systems treat these as separate problems, if they address the everyday case at all.

This project addresses both gaps under Edge AI for Vehicle Health and Predictive Maintenance: a system that is useful on every ordinary drive, and additionally protective in the rare situations where it matters most.

2. Proposed Solution and Value Created

EdgePulse is an edge-cloud cooperative reasoning system that continuously interprets vehicle telemetry through a reasoning model running locally on the vehicle's onboard compute, paired with a larger cloud model that performs asynchronous verification and drives offline improvement.

How it works: A lightweight local model runs continuously on the vehicle, ingesting live telemetry — engine, battery, brake, load, driving-behavior data — and reasoning over relationships and trends

across signals rather than isolated values. When it identifies a meaningful pattern — a slow wear trend or an acute multi-symptom anomaly — it produces a judgment and, where appropriate, a recommended action immediately, with zero dependency on connectivity. When the vehicle is online, a larger cloud model independently verifies the local model's judgment. Disagreements are logged and used to improve future versions of the local model offline, without interrupting vehicle operation. The architecture is inspired by speculative decoding: a lightweight model produces an immediate result, a larger model verifies it asynchronously — applied here to vehicle-health reasoning rather than text generation.

Two operating tiers, one engine:

- **Predictive Health (everyday use):** driving-style-aware maintenance scheduling, component wear-trend forecasting, personalized efficiency coaching, plain-language explanations of dashboard signals in context.
- **Critical Pattern Detection (rare-event use):** multi-signal failure detection that surfaces compounding issues before breakdown, continuing to operate fully offline in exactly the conditions where such failures are hardest to catch by other means.

Explainable reasoning: Unlike conventional diagnostic systems that expose only warning codes, EdgePulse explains the reasoning behind each recommendation — which sensor relationships contributed to a diagnosis. This builds driver trust and gives technicians and fleet operators visibility into why a recommendation exists, not just what it is.

Why this is defensible: Most production diagnostic systems remain rule-based or component-specific. EdgePulse functions as a continuous reasoning layer, independent of any conversational interface, that complements existing systems rather than replacing certified safety systems — positioned as an advisory and predictive layer, with a clear path toward deeper integration as it's validated.

Value created: Drivers get maintenance guidance grounded in actual usage instead of generic intervals. Fleet operators get fleet-wide pattern visibility for cost and risk reduction. Every vehicle gains a reasoning layer that keeps functioning exactly when cloud-based systems can't reach it.

3. Technologies and Frameworks Used

- **Local (edge) reasoning layer:** A lightweight, quantized language model (TinyLlama-1.1B or Phi-3-mini) served locally via Ollama or llama.cpp, running within roughly 1GB RAM. Structured to reason over JSON-formatted telemetry rather than free-form text, producing structured diagnostic and recommendation outputs.
- **Cloud (verification) layer:** A larger model accessed via a high-throughput inference API (Groq or Together AI, running models such as Llama 3.1 70B) performs asynchronous verification of local diagnoses. This layer sits outside the critical decision path — the vehicle never waits on it to act — and audits local judgments when connectivity is available, logging disagreements for offline model evaluation and improvement.

- **Orchestration:** A FastAPI-based orchestrator manages the continuous reasoning loop — ingesting the telemetry stream, invoking the local model, and forwarding context to the cloud model for comparison when connectivity permits. The orchestration is inspired by speculative decoding: a lightweight local model produces an immediate diagnosis, a larger cloud model verifies it asynchronously.
- **Data layer:** A synthetic vehicle telemetry generator producing realistic OBD/sensor-style data with injectable failure and wear patterns, used for both development and live demonstration, enabling controlled, repeatable scenarios on demand.
- **Interface:** A web dashboard showing live telemetry, the local model's running diagnosis, triggered recommendations, and — when online — cloud verification results and logged disagreements.

Why this stack: Every component is free-tier or open-source and runs on a standard consumer laptop (validated at 8GB RAM), while remaining architecturally consistent with what would run on automotive-grade edge compute (an automotive SoC/NPU) in production — making this a genuine proof of concept, not a simulation of one.

4. Key Benefits and Impact

Safety impact: By reasoning over combinations of signals rather than isolated thresholds, EdgePulse is designed to surface early-warning patterns before they'd trigger a conventional dashboard alert, with the goal of converting a roadside breakdown into a scheduled repair. This matters most in low-connectivity zones — rural highways, tunnels, mining and construction sites — where cloud-dependent systems offer no protection today; EdgePulse's local-first architecture allows the reasoning pipeline to continue operating even with limited or no connectivity.

Maintenance cost reduction: Usage-aware scheduling, rather than fixed mileage intervals, allows service timing to reflect actual wear instead of a generic estimate. For fleet operators managing large vehicle populations, reducing unnecessary early servicing while catching preventable major repairs earlier can meaningfully lower operating costs — a single avoided major component failure typically outweighs years of predictive-maintenance tooling cost.

Reduced cloud dependence: Because most routine reasoning happens locally, cloud verification is needed only periodically rather than continuously — reducing network dependence and long-term cloud inference cost relative to a cloud-only diagnostic system.

Everyday relevance: Unlike systems that only activate during failure events, EdgePulse delivers value on every drive — efficiency coaching, wear-trend forecasting, plain-language explanations of vehicle state — so the system is continuously exercised and continuously improved through logged real-world disagreements, rather than sitting dormant until a rare emergency.

Scalability: The same reasoning engine that runs on one vehicle extends naturally to fleet-level pattern detection, where cross-vehicle trends can inform manufacturer-level design feedback and recall-pattern identification — extending the value from individual drivers to OEMs and fleet operators

alike.